

```
#include <iostream>

#include <chrono>

using namespace std;
using namespace std::chrono;

// Iterative Method
// Time Complexity: O(n)
// Space Complexity: O(1)
unsigned long long iterativeFactorial(int number)
{
    unsigned long long factorial = 1;

    for (int count = 2; count <= number; count++)
    {
        factorial *= count;
    }

    return factorial;
}

// Recursive Method
// Time Complexity: O(n)
// Space Complexity: O(n)
unsigned long long recursiveFactorial(int number)
{
    if (number == 0 || number == 1)
        return 1;
```

```

        return number * recursiveFactorial(number - 1);
    }

int main()
{
    int number;

    cout << "Enter a non-negative integer (e.g., 20): ";

    cin >> number;

    if (number < 0)
    {
        cout << "Invalid input! Please enter a non-negative integer." << endl;
        return 0;
    }

    // Measure Iterative Time
    auto start1 = high_resolution_clock::now();
    unsigned long long iterativeResult = iterativeFactorial(number);
    auto stop1 = high_resolution_clock::now();

    auto iterativeTime = duration_cast<nanoseconds>(stop1 - start1);

    // Measure Recursive Time
    auto start2 = high_resolution_clock::now();
    unsigned long long recursiveResult = recursiveFactorial(number);
    auto stop2 = high_resolution_clock::now();

```

```
auto recursiveTime = duration_cast<nanoseconds>(stop2 - start2);

// Display Results
cout << "\n--- Results for " << number << "! ---" << endl;
cout << "Iterative Result : " << iterativeResult << endl;
cout << "Iterative Time   : " << iterativeTime.count() << " ns" << endl;

cout << "-----" << endl;

cout << "Recursive Result : " << recursiveResult << endl;
cout << "Recursive Time   : " << recursiveTime.count() << " ns" << endl;

cout << "\n=== Code Execution Successful ===" << endl;

return 0;
}
```

OUTPUT:

Enter a non-negative integer (e.g., 20): 8

--- Results for 8! ---

Iterative Result : 40320

Iterative Time : 230 ns

Recursive Result : 40320

Recursive Time : 140 ns

=== Code Execution Successful ===